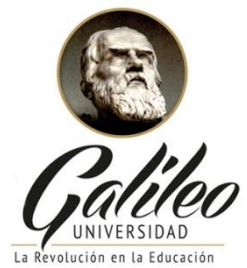


Universidad Galileo	Fecha 4/09/2026
Facultad: FISSIC	Alumno: Pablo Urbina
Curso: Microprocesadores	Carnet: 24001508
Sección: A	Hora de Laboratorio: 1:00 PM
Auxiliar:	Día de Laboratorio: Lunes
Brandon Eduardo Mendoza Espinoza	



Laboratorio # 3 Programación en C

Objetivos:

Aprender a programar en C utilizando funciones enviando argumentos por referencia, manipular punteros, y utilizar doble punteros para trabajar con la memoria teniendo un mayor control sobre la misma, comprendiendo con exactitud como manipular datos por medio de punteros.

Resumen:

Se programaron dos funciones las cuales recibían argumentos por referencia, una siendo la función **int myGCD**, la cual calculaba el MCD de un conjunto de números enteros que se pasan como una cadena de caracteres. La segunda función fue **void myCombinations**, la cual realizaba combinaciones en base a una cadena de caracteres que describía un conjunto de elementos, dichas combinaciones contaban con **n** elementos, donde **n** debía ser menor o igual a 3, a su vez, la cardinalidad del conjunto de elementos debía ser menor o igual a 5.

En ambas funciones era necesario parsear la cadena de caracteres para obtener los datos correctos con los que se iba a trabajar, estos datos se conseguían con el uso de punteros y doble punteros.

Teoría:

El lenguaje C proporciona acceso directo a la arquitectura del procesador y la memoria física RAM, lo cual significa que el programador tiene un control mucho más amplio sobre la memoria que el control que proporcionan otros lenguajes de programación. Los punteros son una herramienta esencial para esto, se definen como variables cuya función principal es almacenar direcciones de memoria en lugar de valores literales.

Por definición, C realiza el paso de argumentos a funciones por valor, creando copias locales de las variables en el stack frame de la función llamada, así que para lograr el comportamiento de un paso por referencia, se envían como argumento las direcciones de memoria de las variables, es decir, punteros, lo que concede a la función la capacidad de acceder a la dirección original y modificar su contenido de forma persistente fuera de su propio ámbito local, lo cual es fundamental cuando se deben retornar múltiples valores o cuando se transfieren estructuras de datos grandes.

Un puntero doble es una variable que almacena la dirección de memoria de otro puntero simple, en un contexto de procesamiento de cadenas y arreglos multidimensionales, los punteros dobles permiten construir tablas dinámicas o estáticas de direcciones por medio de una arquitectura de 3 niveles: Un búfer físico de datos, un arreglo de punteros locales y una referencia de doble puntero. Esto permite realizar búsquedas y operaciones de indexación con aritmética de punteros sin tener que duplicar información ni depender de asignación dinámica en el heap.

El parsing sin librerías se realizó analizando secuencialmente la memoria mediante punteros de lectura y comprobaciones ASCII.

El Máximo Común Divisor es el número entero positivo más grande que divide exactamente a dos o más números sin dejar residuo, el método computacional más eficiente para calcularlo es el método de Euclides, el cual se basa en la propiedad de divisiones sucesivas utilizando el operador modulo con una complejidad temporal logarítmica.

Datos Prácticos:

```
static const char numbers[] = "36, 60, 48";
int main(void){
    int result = myGCD(set: numbers);
    char comb[100];

    printf("Resultado MCD: %d\n", result);
[model] Resultado MCD: 12
```

Resultado de la función myGCD

```
// Pruebas para myCombinations

// Pruebas validas
myCombinations( n:1, set: "A = {rojo, verde, azul}", comb);
myCombinations( n:2, set: "A = {rojo, verde, azul}", comb);
myCombinations( n:3, set: "A = {rojo, verde, azul, amarillo, negro}", comb);

// Pruebas invalidas
myCombinations( n:4, set: "A = {rojo, verde, azul}", comb); // n mayor a 3
myCombinations( n:1, set: "A = {rojo, verde, azul, amarillo, negro, blanco}", comb);
myCombinations( n:10, set: "A = {rojo, verde, azul, amarillo, negro, blanco}", comb);
[model] Combinaciones: {rojo}, {verde}, {azul}
[model] Combinaciones: {rojo, verde}, {rojo, azul}, {verde, azul}
[model] Combinaciones: {rojo, verde, azul}, {rojo, verde, amarillo}, {rojo, verde, negro}, {rojo, azul, amarillo}, {rojo, azul, negro}, {rojo, amarillo, negro}, {verde, azul, amarillo}, {verde, azul, negro}, {verde, amarillo, negro}, {azul, amarillo, negro}
[model] n debe ser menor o igual a 3
[model] La cardinalidad de set debe ser menor o igual a 5
[model] n debe ser menor o igual a 3
```

Resultado de la función myCombinations en diferentes escenarios

Cálculos Teóricos:

```
// Parsear la cadena para extraer los números enteros
int numero = 0;
while(*ptr >= '0' && *ptr <= '9'){
    numero = (numero * 10) + (*ptr - '0');
    ptr++;
}

if (mod != 0){
    while(numero != 0){
        int temp = mod % numero;
        mod = numero;
        numero = temp;
    }
} else {
    mod = numero;
}
```

Conversión a entero de cada número (36, 60, 48)

$$'0' = 48$$

36 (carácter 3, ASCII 51):

$$\text{numero} = (0 \times 10) + (51 - 48) = 0 + 3 = 3$$

36 (carácter 6, ASCII 54):

$$\text{numero} = (3 \times 10) + (54 - 48) = 30 + 6 = 36$$

Resultado final:

$$\text{numero} = 36$$

60 (carácter 6, ASCII 54):

$$\text{numero} = (0 \times 10) + (54 - 48) = 0 + 6 = 6$$

60 (carácter 0, ASCII 48):

$$\text{numero} = (6 \times 10) + (48 - 48) = 60 + 0 = 60$$

Resultado final:

$$\text{numero} = 60$$

48 (carácter 4, ASCII 52):

$$\text{numero} = (0 \times 10) + (52 - 48) = 0 + 4 = 4$$

48 (carácter 8, ASCII 56):

$$\text{numero} = (4 \times 10) + (56 - 48) = 40 + 8 = 48$$

Resultado final:

$$\text{numero} = 48$$

Cálculo del MCD:

$$\text{MCD}(36,60,48) = \text{MCD}(\text{MCD}(36,60),48)$$

MCD inicial:

$$\text{mcd} = 36$$

Para MCD(36,60)

Iteración 1:

$$\text{Residuo} = 36 \quad (\text{mod } 60) = 36 \quad \text{mcd} = 60, \quad \text{numero} = 36$$

Iteración 2:

$$\text{Residuo} = 60 \quad (\text{mod } 36) = 24 \quad \text{mcd} = 36, \quad \text{numero} = 24$$

Iteración 3:

$$\text{Residuo} = 36 \quad (\text{mod } 24) = 12 \quad \text{mcd} = 24, \quad \text{numero} = 12$$

Iteración 4:

$$\text{Residuo} = 24 \quad (\text{mod } 12) = 0 \quad \text{mcd} = 12, \quad \text{numero} = 0$$

Condición de parada, resultado parcial:

$$\text{mcd} = 12$$

Para MCD(12,48)

Iteración 1:

Residuo = 12 (mod 48) = 12 mcd = 48, numero = 12

Iteración 2:

Residuo = 48 (mod 12) = 0 mcd = 12, numero = 0

Condición de parada, resultado final:

mcd = 12

Conclusiones:

El uso de punteros en C es crucial debido a que permite tener un control amplio sobre la memoria con la que trabajamos, si bien en una computadora los recursos son enormes hasta el punto que “podemos” desperdiciar memoria realizando operaciones sin preocuparnos de experimentar una pérdida de velocidad notable en nuestro programa u otras consecuencias, al trabajar en un sistema embebido los recursos son muchísimo más limitados, por lo que un manejo eficaz de la memoria es crucial para que el rendimiento de la aplicación sea rápido y eficaz para cumplir su tarea.

Es importante utilizar punteros de forma correcta para manipular la memoria como sea necesario para obtener la mayor eficiencia posible, a su vez, hay que estar familiarizado con el sistema embebido que se va a utilizar para el programa antes de programarlo, esto es para conocer que recursos tenemos y planificar como se van a manejar de antemano.

Bibliografía:

[1] GeeksforGeeks, "Pointers in C," GeeksforGeeks, 2024.

<https://www.geeksforgeeks.org/c-pointers/>

[2] Tutorialspoint, "C - Pointers," Tutorialspoint, 2024.

https://www.tutorialspoint.com/cprogramming/c_pointers.htm

[3] Wolfram MathWorld, "Euclidean Algorithm," Wolfram MathWorld, 2024.

<https://mathworld.wolfram.com/EuclideanAlgorithm.html>